

Chatroom distribuée SOAP + REST

Architecture polyglotte avec frontend hexagonal

Groupe 3

École Supérieure Polytechnique — DGI

Cours Web Services — DIC3 — Mai 2026

Plan

1. Sujet et fonctionnalités
2. Conception et architecture
3. Modèle de données et intégrité
4. Communication SOAP
5. Communication REST
6. Bilan

Sujet

Le cas pratique du cours : une chatroom en SOAP et en REST.

Fonctionnalités

S'inscrire

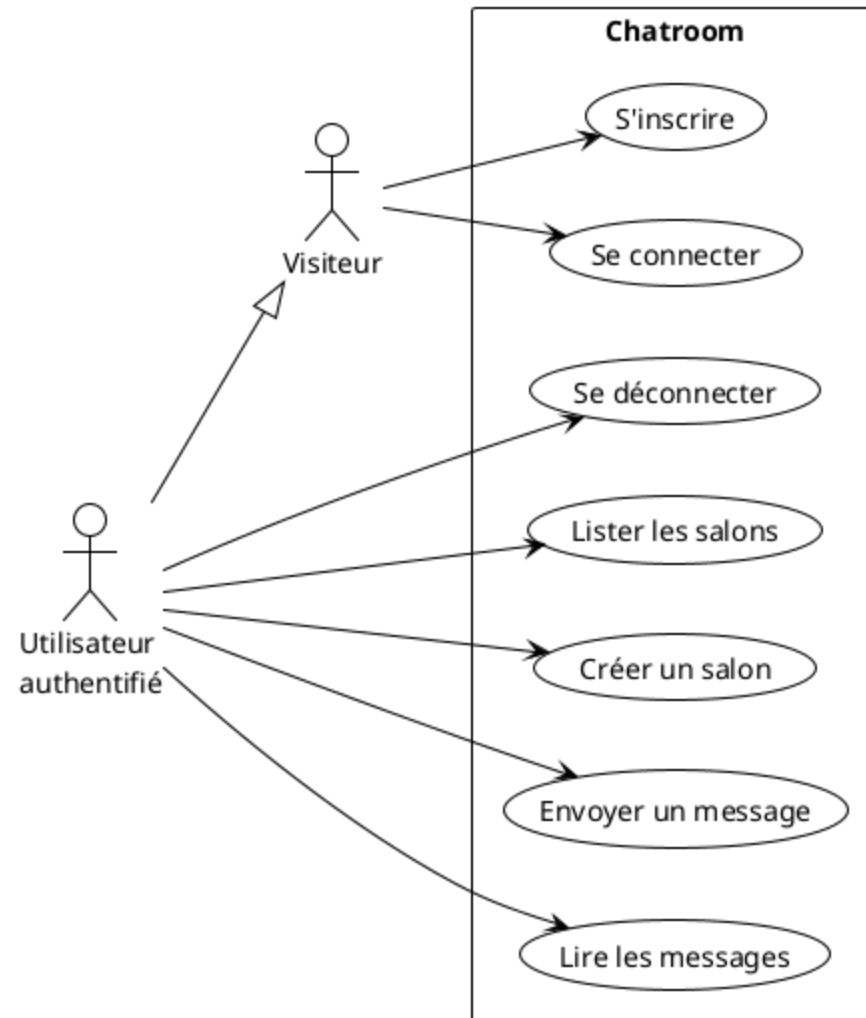
Se connecter

Créer ou rejoindre un salon

Envoyer un message

Voir les messages des autres en temps réel

Se déconnecter



Architecture

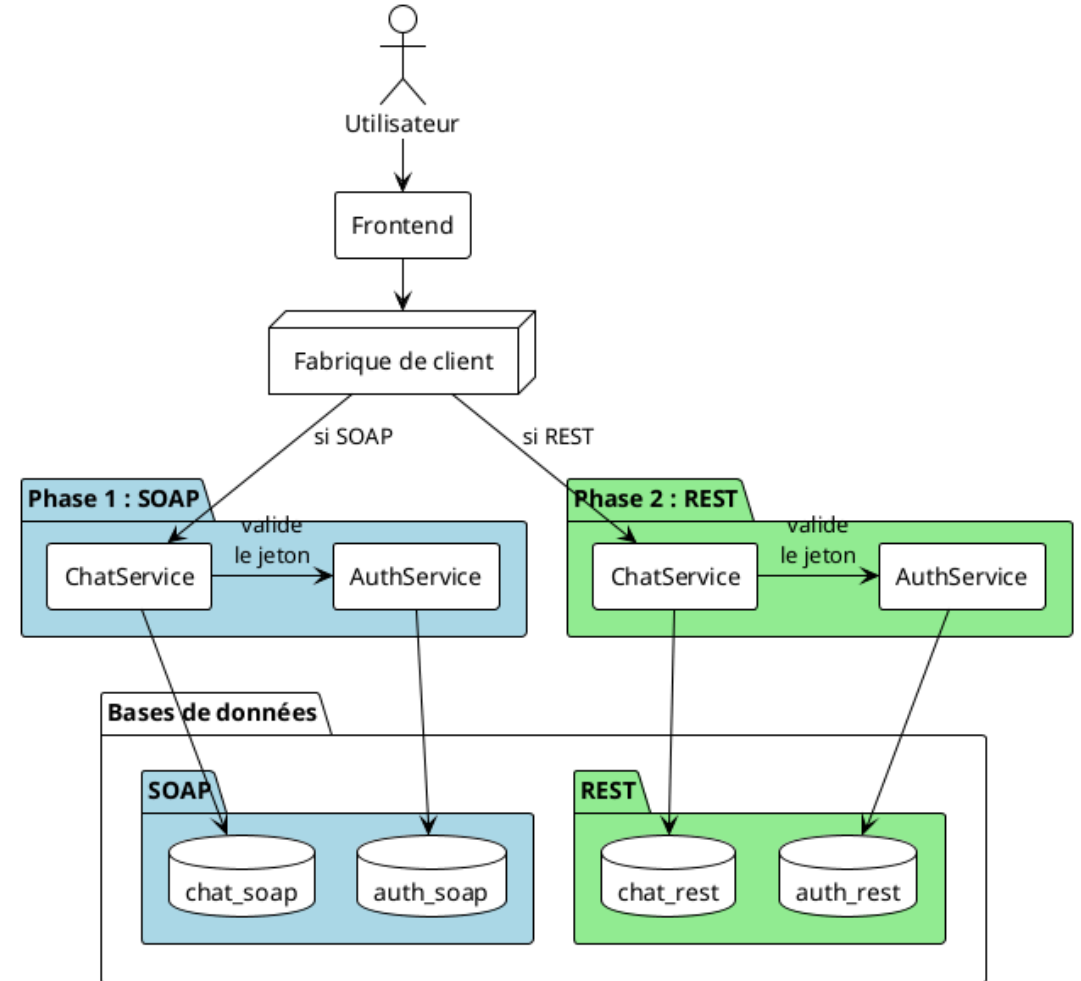
Un même frontend pour deux protocoles

Deux jeux de services backend interchangeable

Validation du jeton à chaque opération

Bases de données isolées par phase

Bascule par variable d'environnement

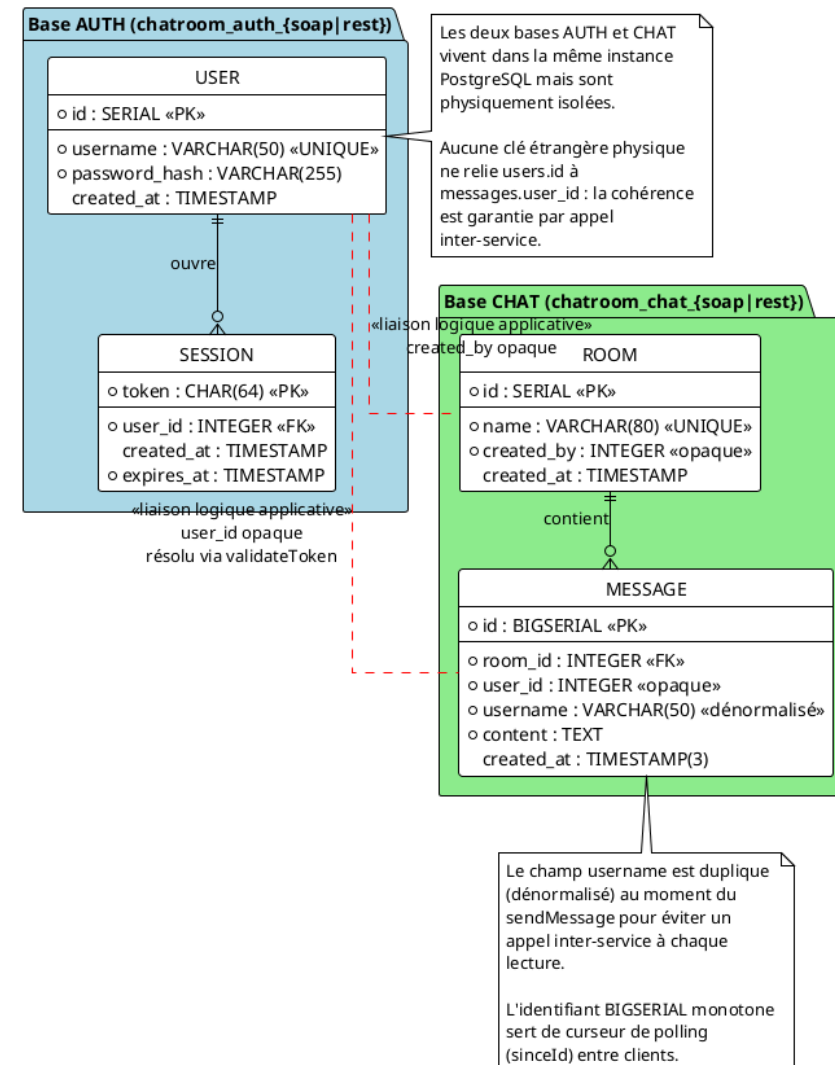


Structuration des données

Comment garantir l'intégrité sans clé étrangère ?

Les deux bases d'un même backend sont **physiquement isolées**. PostgreSQL n'autorise pas de clé étrangère inter-base.

Modèle Conceptuel de Données (MCD) — backends isolés

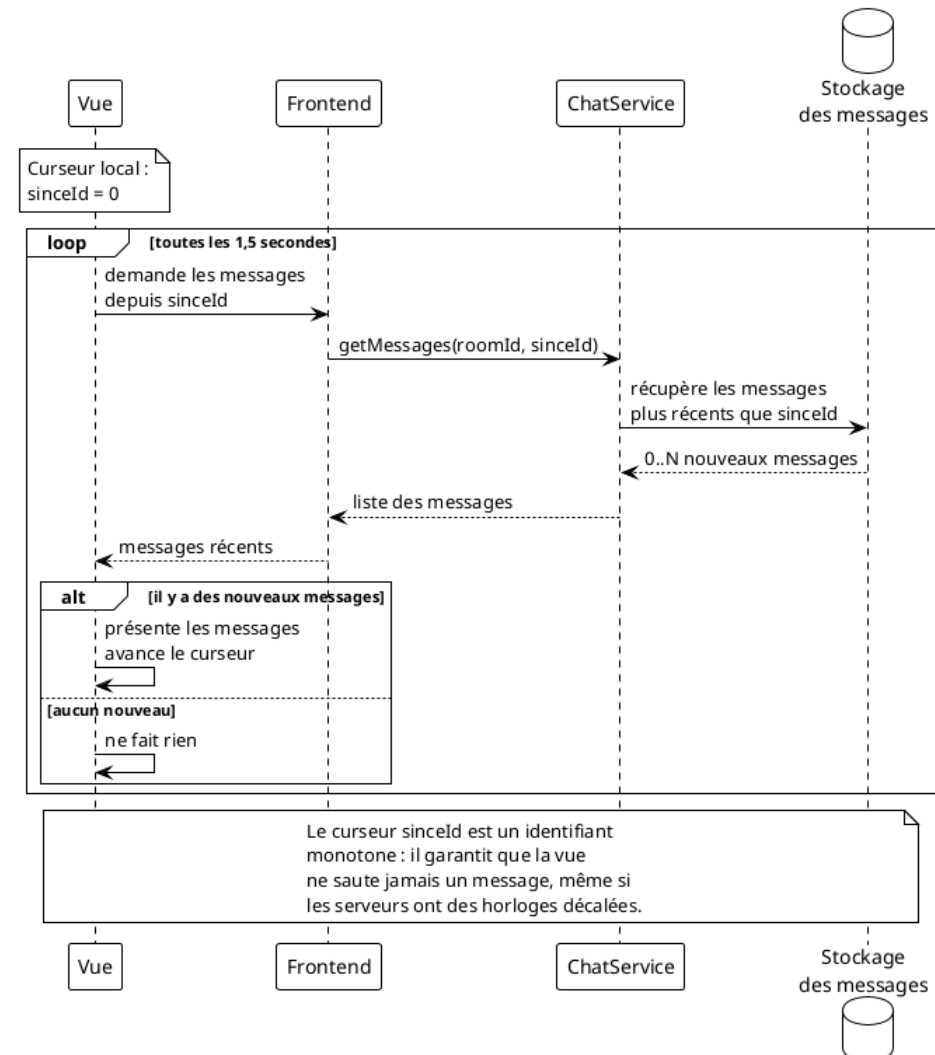


Polling côté vue

Comment livrer les nouveaux messages en temps réel ?

La vue interroge le serveur toutes les **1,5 secondes** et utilise un curseur `sinceId` monotone pour ne recevoir que les messages plus récents.

Séquence du polling côté vue



Stack technique

PHP 8.2 côté frontend

SoapClient et **cURL** dans la fabrique de client

Java 21 pour les deux AuthService

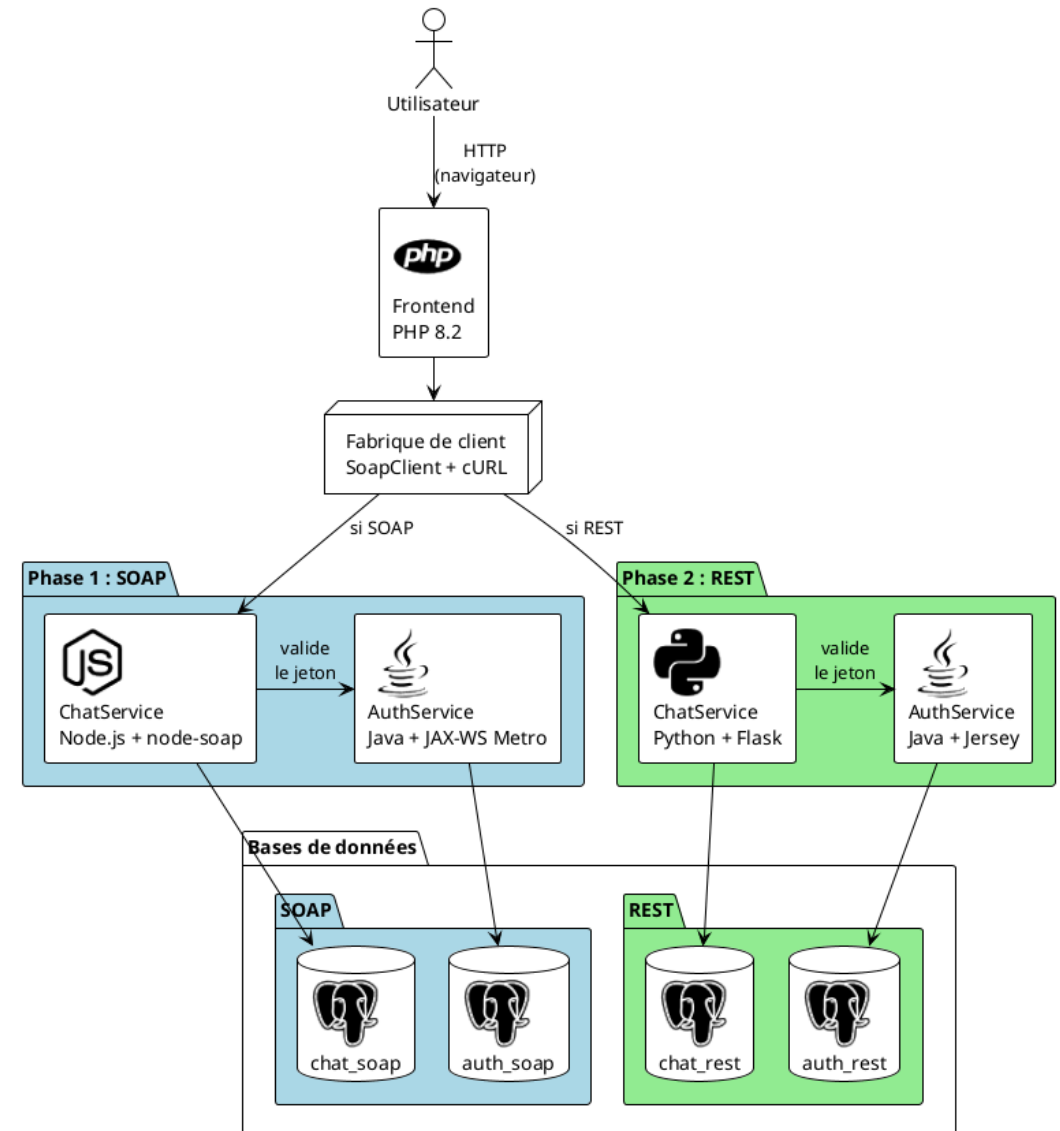
Node.js 20 pour le ChatService SOAP

Python 3.12 pour le ChatService REST

PostgreSQL 16 pour les quatre bases

bcrypt pour les mots de passe

HTTP entre le navigateur et le frontend



Communication

Comment les services dialoguent

SOAP : zoom sur l'architecture

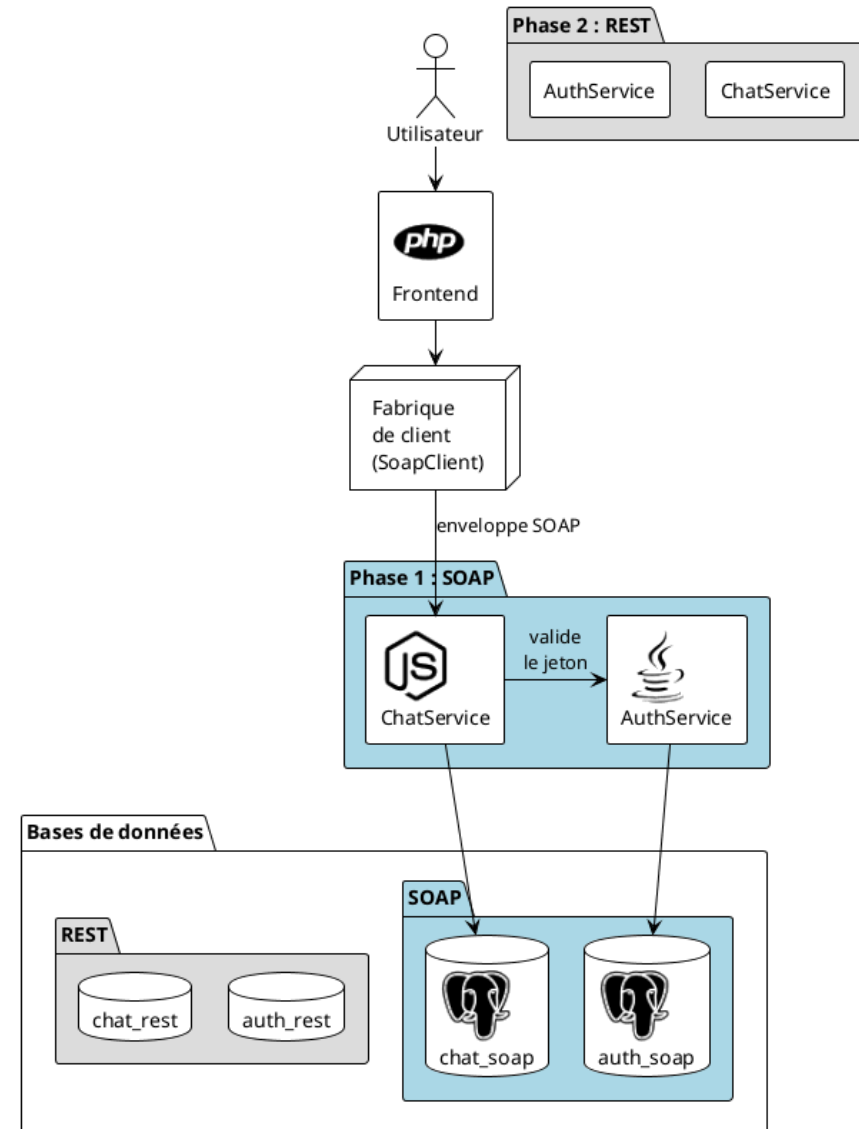
Contrat **WSDL** décrivant les opérations

Échanges en **enveloppes XML**

Style **document/literal**

Fautes structurées (`<soap:Fault>`)

Génération auto des stubs côté PHP et Node



Une enveloppe SOAP en pratique

Requête login

```
<soap:Envelope>
  <soap:Body>
    <auth:login>
      <username>alice</username>
      <password>secret</password>
    </auth:login>
  </soap:Body>
</soap:Envelope>
```

Réponse

```
<soap:Envelope>
  <soap:Body>
    <auth:loginResponse>
      <return>abc123...</return>
    </auth:loginResponse>
  </soap:Body>
</soap:Envelope>
```

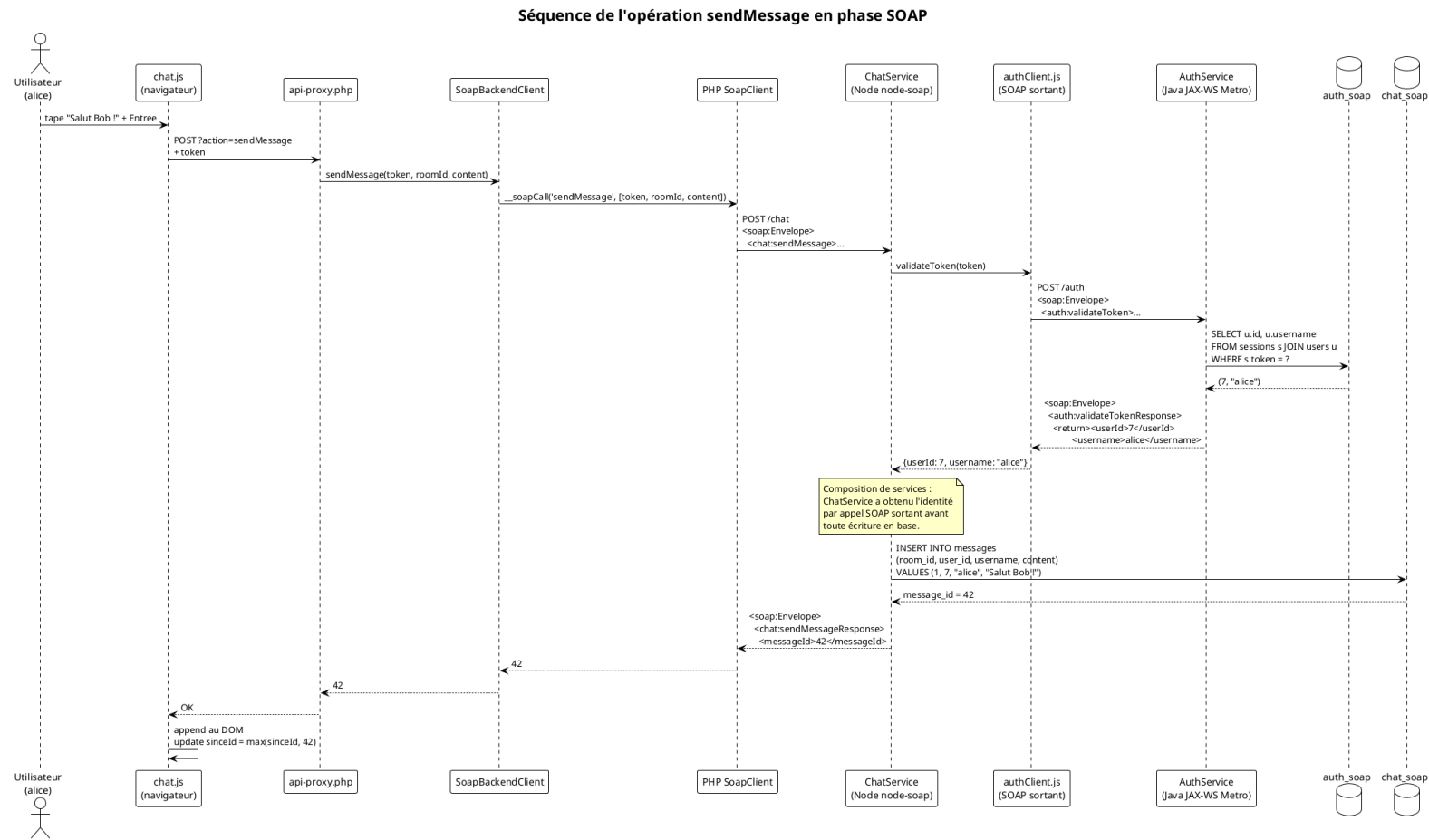
Codes d'erreur : INVALID_CREDENTIALS ,
USER_ALREADY_EXISTS , INVALID_TOKEN

Nos deux WSDL

Chaque service SOAP du projet expose son WSDL ; voici les opérations qu'il rend disponibles.

AuthService — http://localhost:9001/auth	ChatService — http://localhost:9002/chat
<code>register(username, password)</code>	<code>createRoom(token, name)</code>
<code>login(username, password)</code>	<code>listRooms(token)</code>
<code>logout(token)</code>	<code>sendMessage(token, roomId, content)</code>
<code>validateToken(token)</code>	<code>getMessages(token, roomId, sinceId)</code>

SOAP : envoi d'un message



SoapUI en action

14 cas de test chaînés par Property Transfers

23 assertions vérifiées automatiquement

Inspection du **XML brut** (onglet Raw)

REST : zoom sur l'architecture

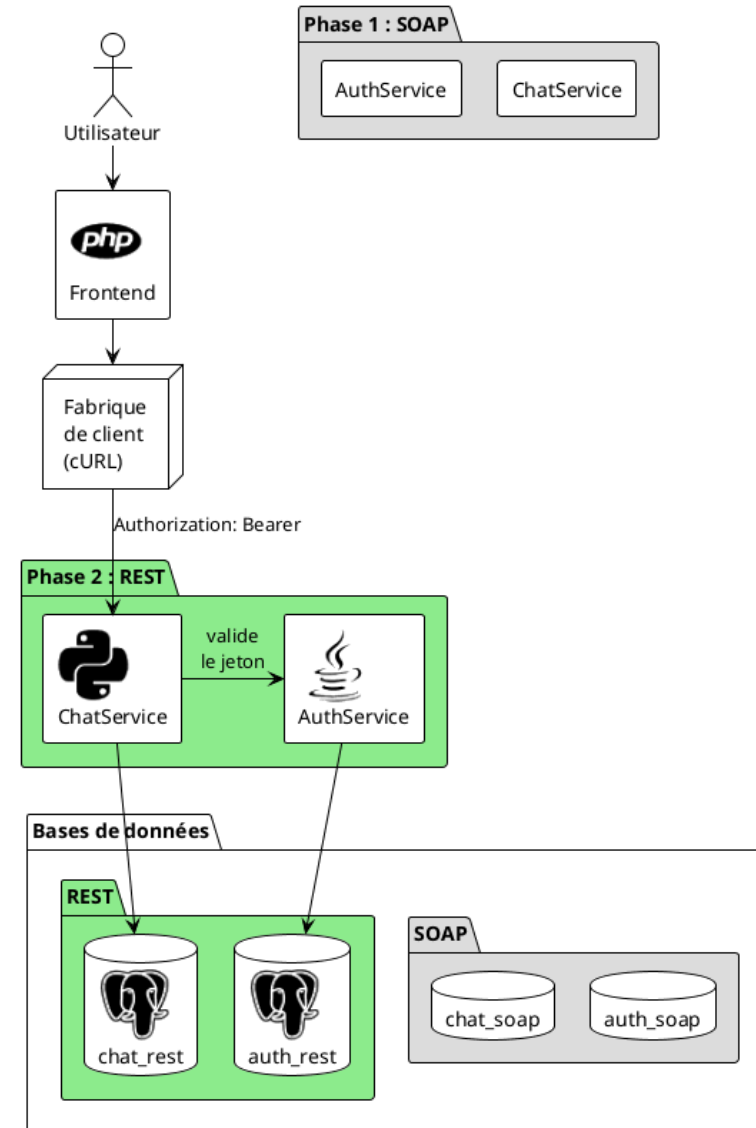
Ressources `/rooms` , `/rooms/{id}/messages`

Verbes HTTP : `GET` , `POST`

Auth par en-tête `Authorization: Bearer <token>`

Négociation **JSON** ou **XML** via `Accept`

Pas d'état conversationnel



Nos endpoints REST en pratique

Le projet expose **huit endpoints métier**, répartis entre les deux services.

Verbe + chemin	Service	Code OK
POST /register	AuthService	200
POST /login	AuthService	200
GET /validate	AuthService	200
POST /logout	AuthService	204
GET /rooms	ChatService	200
POST /rooms	ChatService	201
GET /rooms/{id}/messages	ChatService	200
POST /rooms/{id}/messages	ChatService	201

Négociation de contenu en action

Même requête, format JSON

```
curl -H 'Accept: application/json' \  
http://localhost:8082/rooms
```

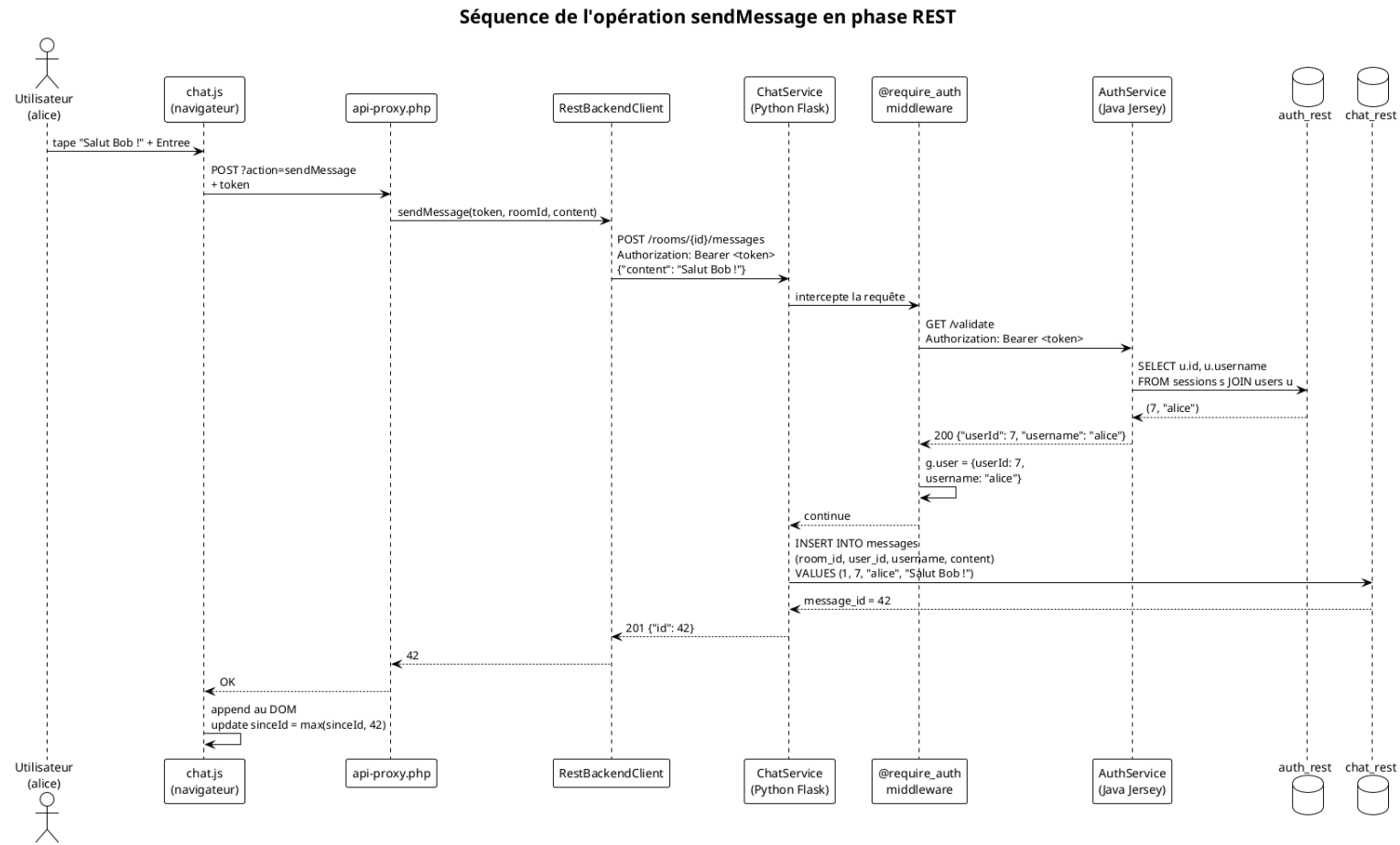
```
{"rooms": [  
  {"id": 1, "name": "general"}  
]}
```

Même requête, format XML

```
curl -H 'Accept: application/xml' \  
http://localhost:8082/rooms
```

```
<response>  
  <rooms>  
    <room>  
      <id>1</id>  
      <name>general</name>  
    </room>  
  </rooms>  
</response>
```

REST : envoi d'un message



Postman en action

Collection organisée par service

28 requêtes : JSON et XML pour chaque opération

Property Transfers automatiques (token, roomId, sinceId)

Rejouable via Runner ou **Newman CLI**

Bilan

- **Composition de services** observable : chaque opération métier traverse les deux services et illustre la délégation de l'authentification.
- **Polyglossie tenue** : Java, Node.js et Python interagissent sans friction grâce aux contrats standardisés.
- **Bascule SOAP**

REST par simple variable d'environnement, sans modification du code applicatif, grâce au pattern **Port et Adaptateurs**.

Merci de votre attention.